

DATA SCIENCE & MACHINE LEARNING CHALLENGE

Optimización de Abastecimiento

Pronóstico de demanda por SKU-Tienda + optimización de pedido bajo incertidumbre

Caso A · Retail — Tostaó

Juliana Beltrán · Lead Data Scientist

Presentación Ejecutiva

El reto de negocio

Predecir demanda es solo la mitad de la batalla



Stockout

Pedir menos de lo necesario: se pierden ventas y el margen asociado a esa unidad no vendida.



Overstock

Pedir más de lo necesario: se incurre en costo de almacenamiento y capital inmovilizado.



Objetivo

Pronosticar demanda semanal por SKU-Tienda y determinar la cantidad de pedido que minimiza el costo total esperado, balanceando ambos riesgos.

Análisis exploratorio

Qué nos dicen los datos antes de modelar



Ventana de datos limitada

~4 meses de historial de ventas por SKU-Tienda — condiciona la capacidad de capturar estacionalidad completa.



Series por SKU-Tienda

Cada combinación tienda-producto tiene su propio patrón de demanda; se requieren variables de rezago (lags) y ventanas móviles.



Márgenes muy superiores al costo de guardar

El costo de almacenamiento semanal es órdenes de magnitud menor que el margen unitario en la mayoría de productos.

Enfoque metodológico

Cuatro modelos evaluados, validación respetando el tiempo

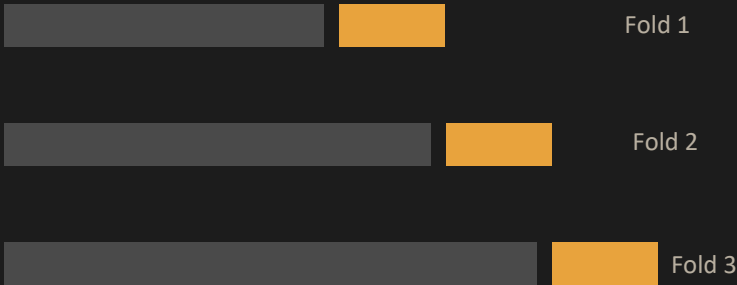
Modelo	Tipo	Resultado
Linear Regression	Lineal (baseline)	Referencia
Random Forest	Ensamble de árboles	Buen desempeño
Extra Trees	Ensamble de árboles (extra-randomized)	★ Mejor modelo
Gradient Boosting	Boosting secuencial	Buen desempeño

¿Por qué TimeSeriesSplit y no CV aleatorio?

- Un k-fold aleatorio mezclaría datos futuros dentro del set de entrenamiento (fuga de información temporal).
- TimeSeriesSplit (5 folds) entrena siempre con el pasado y valida con el futuro, replicando el escenario real de producción.
- Los hiperparámetros finales se afinaron con GridSearchCV sobre esos mismos folds.



Validación temporal



■ Train ■ Test

El corte de test siempre respeta el orden cronológico del histórico de ventas.

Resultados del pronóstico

ExtraTreesRegressor, optimizado con GridSearchCV

0.70

R^2 en test

90%

Cobertura del intervalo
de confianza (calibrado)

5

Folds de
TimeSeriesSplit



Limitación reconocida: con solo ~4 meses de historia, el modelo no alcanza a capturar estacionalidad completa (ej. festivos, temporadas altas). El R^2 de 0.70 es sólido dado ese contexto, no el techo del enfoque.



Iteración de rigor metodológico: el primer enfoque de intervalos (dispersión entre árboles) dio solo 52% de cobertura frente al 90% objetivo — un problema conocido en ensembles tipo bagging. Se recalibró usando los residuales del set de validación, alcanzando 90.00% de cobertura.

De la incertidumbre a la decisión

Enfoque tipo newsvendor: el margen decide qué tan agresivo pedir

Límite inferior
(demanda_lower)

Límite superior
(demanda_upper)



Demanda
objetivo



Ratio de criticidad

$$\text{costo_stockout} / (\text{costo_stockout} + \text{costo_overstock})$$

- Ratio cercano a 1 → postura agresiva (pedir cerca del límite superior).
- Ratio cercano a 0 → postura conservadora (pedir cerca del límite inferior).
- La demanda objetivo se interpola dentro del intervalo según ese ratio.



Asignación final (PuLP)

Minimiza: costo total de almacenamiento

Sujeto a: stock actual + pedido $\geq$ demanda objetivo

Se resuelve como programación lineal entera, por SKU-Tienda.

Impacto de negocio estimado

160 combinaciones SKU-Tienda evaluadas



ROI del buffer
de seguridad

~97x

Por cada \$1 adicional en costo de almacenamiento del buffer, se protegen ~\$97 en margen ante escenarios de demanda alta.

Métrica	Valor
Margen protegido por el buffer	\$18.301.842
Costo de mantener el buffer	\$189.555
Costo almacenamiento (política optimizada)	\$485.130
Costo almacenamiento (política naive, sin buffer)	\$294.370
Costo incremental del "seguro"	\$190.760



Hallazgo clave: el ratio de criticidad promedio es 0.99 — el margen por unidad supera ampliamente el costo de almacenamiento en este catálogo, por lo que el modelo prioriza consistentemente evitar quedarse sin stock.

Conclusiones y próximos pasos



Conclusiones

- ExtraTreesRegressor con TimeSeriesSplit + GridSearch logra $R^2=0.70$ pese a la ventana corta de datos.
- El intervalo de confianza, recalibrado con residuales, alcanza 90% de cobertura real.
- El enfoque newsvendor conecta directamente la incertidumbre del modelo con la decisión de pedido, usando el margen de cada SKU como guía.
- El costo incremental del buffer de seguridad (\$190K) es marginal frente al margen que protege (~\$18.3M).



Próximos pasos

- Incorporar mayor historial (12+ meses) para capturar estacionalidad y festivos.
- Sumar variables exógenas: promociones, clima, tráfico por tienda.
- Calibrar el intervalo con un set de validación independiente del test final.
- Monitorear el modelo en producción: drift de demanda y recalibración periódica del intervalo.

Gracias

Repositorio: github.com/julibeltran19/PRUEBA_TOSTAO_ABASTECIMIENTO

Preguntas y discusión